Compatibility, status, and extension

ar090 · 14 August 2026 · Draft

Contents

Developer guide

Compatibility

snnlang is additive. Historical commands and experiments use the legacy executor unless they explicitly select the graph executor. Bundles remain data-only, and *tools/snn* does not import the authoring package when replaying them.

Migration requires layered comparisons of structure, parameters, initialization, forward state, gradients, optimizer updates, checkpoints, resumed trajectories, learning curves, downstream measurements, and publication outputs. A successful smoke test does not establish equivalence.

Issue 73 is the active implementation and migration checklist. The independent exp022 gold-star campaign remains outside this documentation work.

Current support

Area	Status	Current boundary
Graph authoring	Implemented	Typed populations, inputs, parameters, projections, groups, outputs, and observables.
Bundles	Implemented	Canonical JSON, hashes, manifests, assets, reports, and diagrams.
Graph simulation	Implemented	Dense COBA-LIF and leaky-integrator graphs with AMPA/GABA projections.
Arbitrary topology	Implemented	Named feedforward, recurrent, and feedback projections with integral delays.
Runtime continuation	Implemented	Validated save, load, and continuation of dynamic graph state.
Readouts	Implemented	Mean voltage, final voltage, spike count, spike rate, cumulative potential, and valid-

		time masks execute through graph operations.
Input bindings	Implemented	Dense arrays, valid-time masks, sparse event streams, fixed/categorical Poisson, and portable dense/prebinned/ timestamped-event dataset snapshots emit versioned execution protocols.
Training recipes	Implemented	Validated objectives, regularizers, groups, gradient choices, duration, and reachability compile to canonical data.
Native training	Core implemented	Deterministic epoch/minibatch AdamW training and versioned named checkpoints support exact mid-epoch CPU resume; accelerator stochastic state and parity gates remain.
Collection migration	Not implemented	Requires the complete issue 73 equivalence process.

What remains
The following work is **not implemented**. Issue 73 tracks it as an active checklist.

Complete graph and protocol vocabulary
Portable immutable NPZ dataset snapshots now cover feature-scaled rate-Poisson encoding, exact prebinned spikes, and graph-timestep binning of timestamped events. Deterministic selection and target binding support MNIST-like dense and SHD-like event data without a hidden dataset registry or download side effect. The complete collection training vocabulary is implemented: disabled and trainable recurrent variants, initializer metadata, constraints and units, exhaustive parameter groups, gradient choices, spike-budget regularization, variable graph timesteps and presentation durations, differentiable-route validation, and element-level rejection are explicit. Dataset identity, split, source digest and keys, total/ selected indices, sample cap, batch size, shuffle behavior, duration, encoder configuration, and execution/order/encoder seeds are recorded in the execution protocol.

Graph-native training and checkpoints
Deterministic dataset iteration, named digest-bearing target bindings, versioned graph-training checkpoints, selected and final persistence, strict named loading, the one-layer legacy parameter map, and exact mid-epoch CPU optimizer/random-stream/data-order resume are implemented.

- Extend the checkpoint contract to accelerator stochastic state when accelerator training is validated.

A compact local production matrix now executes MNIST-shaped one-layer PING and SHD-shaped three-layer PING graphs, trains every named parameter in the deep recurrent SHD recipe, and combines categorical variable-rate input with fine-timestep inference. It verifies public output/recording shapes and complete recurrent gradient coverage without

downloading datasets. Full dataset trajectories, accelerator parity, and campaign-scale resource/job shapes remain campaign gates.

Inference and interventions

Portable selected/final graph checkpoint loading for simulation and inference is implemented with exact graph/name/shape/dtype validation and checkpoint provenance in metrics. A versioned, request-local override contract supports Poisson duration and rate plus scaling of named projections. It rejects ambiguous input modes, invalid values, and unknown projections; timestep replacement remains unsupported because it requires graph recompilation rather than runtime mutation. Inference timestep changes now recompile an immutable graph copy and rebuild physical decay and delay planning. Only generated Poisson bindings are resampled; their physical presentation duration is preserved unless separately overridden. Checkpoints authenticate against the source graph before their same-shaped named parameters load into the effective graph. Dense/event replay, incompatible delays, and runtime-state conversion fail closed. Named hidden-population spike deletion and Poisson-addition interventions are implemented as an ordered, seeded, request-local contract. Intervened spikes feed later zero-delay populations, delayed histories, recordings, and readouts through the ordinary graph schedule without backend callbacks. Graph CLI inference now writes a versioned manifest that inventories the stable names, shapes, and data types in output, recording, and parameter payloads. It binds their content digests to graph, seed, execution protocol, checkpoint, override, intervention, recording, and device provenance; validation fails closed on identity drift or corruption. A derived-inference contract resolves exact public logits and population-spike names into labels, predictions, accuracy, per-presentation per-cell rates, and sparse time/batch/cell rasters while retaining the authenticated source-artifact digest.

Artifacts and campaigns

Metrics, checkpoints, inference artifacts, and runstore provenance now use versioned schemas. Every newly initialized run stores an explicit `legacy` or `graph` executor. Graph runs require a prefixed graph digest and may record the training digest; legacy runs reject graph identities. Inventory, archive, verification, and restore retain the manifest, while promotion carries executor and graph/training identities into reverse provenance without changing campaign ownership.

- Wire the collection orchestrator's future graph campaign plan to the explicit runstore fields after the independent legacy campaign is complete.
- Validate resumption and promotion through a complete graph campaign worktree without changing the active publication view prematurely.

Conformance and migration

The first hand-checkable CPU cases prove exact named parameter tensors and complete E/I spike, membrane, AMPA, and GABA traces for both feedforward-isolated and actively recurrent one-layer PING networks built from one parameter set. Mean-voltage logits agree under the predeclared 10^{-6} absolute and relative CPU tolerance. The first case also fixed the legacy recurrent layer map to its actual one-based names and aligned the `MeanVoltage` default with the legacy 2 ms output membrane.

A four-update backward case makes all six feedforward and recurrent tensors trainable and compares the complete loss trajectory, every final named gradient, the constrained final parameters, and all named AdamW tensors under the same predeclared tolerance. A two-update checkpoint followed by two resumed updates is bit-identical to the uninterrupted graph trajectory. The fixture applies the legacy trainer's non-negative projection after each optimizer step, separating optimizer state from the constrained stored parameter value.

Bidirectional parameter interchange now imports and exports the supported legacy state keys with exact coverage, shape, dtype, and mapping-version provenance. Forward conformance is rerun after a graph → legacy → graph round trip. Optimizer interchange remains deliberately excluded because a legacy optimizer object does not satisfy the portable graph checkpoint contract.

A five-sample, two-epoch shuffled dataset case independently reconstructs each seed-derived permutation and uneven minibatch in direct PyTorch. All six update coordinates and losses, plus the final gradient, parameter, and AdamW tensors, conform under the frozen CPU policy. Mid-epoch resume rejects a changed shuffle protocol before restoring state.

The representative production-shaped fixtures add the 784-channel/10-class MNIST contract and the 700-channel/20-class SHD contract. The SHD example is a trainable three-PING hierarchy with exhaustive parameter coverage and a six-population spike budget. Its focused CPU update confirms gradients for feedforward, all recurrent E→I/I→E tensors, and the readout. A separate PING fixture recompiles from 0.1 ms to 0.05 ms while preserving presentation duration and sampling categorical rates across three presentations. These are interface and numerical-path gates, not claims about dataset accuracy.

- Compare topology, parameter tensors, initialized state, forward traces, loss, gradients, optimizer updates, checkpoint interchange, exact resume, learning trajectories, interventions, and aggregations.
- Define exact fields and numerical tolerances before viewing the final comparison.
- Run CPU and publication-accelerator conformance cases and record any limits on numerical determinism.
- After the independent legacy gold-star campaign is complete, run the full snnlang campaign under the frozen scientific protocol and compare every result, uncertainty band, claim, and figure.
- Retain both immutable campaigns and require deliberate review before changing the default executor or migrating the publication collection.

Extending snnlang

Add a feature when it represents reusable computation or execution. Keep it in an experiment runner when its meaning depends on one hypothesis or figure.

A capability should include a clear authoring API, versioned serialization, compiler validation, precise diagnostics, a hand-checkable execution fixture, explicit state and provenance semantics, and compatibility tests. Add an executable example once the feature is mature enough to teach.

Avoid opaque callbacks, access to backend internals, positional checkpoint guesses, and silent fallbacks.

API reference

Validation and diagnostics

```
snn.validate_graph(graph) -> ValidationResult
result.raise_for_errors() -> None
```

`ValidationResult` exposes `.diagnostics`, `.errors`, and `.warnings`. Each `Diagnostic` contains `severity`, `code`, `message`, and optional `subject`; `.line()` renders a stable one-line form.

Compilation raises graph and training errors. Target capability gaps remain diagnostics so a bundle can still describe computation unsupported by the selected backend.

Executor capability inspection

```
graph_capability_issues(graph) -> list[CapabilityIssue]
plan_graph(graph) -> GraphPlan
```

`CapabilityIssue` identifies the graph element, missing capability, and message. `plan_graph` performs execution-specific topology, shape, polarity, scheduling, and integral-delay validation before constructing a `GraphPlan`.

`GRAPH_CAPABILITIES_V1` is the current machine-readable executor boundary. It lists supported neuron, synapse, operation, connection, recording, delay, and training capabilities.

Conformance reports

```
policy = ComparisonPolicy(mode="numeric", atol=1e-6, rtol=1e-5)
report = compare_conformance_layers(
    case_id,
    reference,
    candidate,
    policies={"forward": {"logits": policy}},
)
report.require_passed()
write_conformance_report(path, report)
```

Reports use schema `tools/snn.conformance-report/v1`. Each layer and field is named explicitly. Missing fields, extra fields, shapes, dtypes, values, maximum absolute and relative errors, and the applied policy are recorded. Exact comparison is the default; numerical tolerance must be declared for a specific field, and an unused rule is rejected. `canonical_json_tensor` encodes topology and provenance structures for exact comparison beside numerical tensors.

Inference overrides

```
simulate(ExecutionSpec(
  kind="simulate",
  executor="graph",
  bundle="trained.bundle",
  checkpoint="selected.checkpoint",
  poisson_bindings=(binding,),
  options={"inference_overrides": {
    "duration_ms": 400.0,
    "input_rate_hz": 25.0,
    "projection_scales": {"sensory_ping_I_to_E": 0.5},
  }},
))
```

The contract uses schema `tools/snn.inference-overrides/v1`. Overrides apply after checkpoint loading to one built model only; the graph bundle and checkpoint are not modified. Duration must be a positive integral number of graph timesteps. Rate must be finite and non-negative. Projection scales use exact graph projection identifiers and finite non-negative factors. Duration and rate overrides require generated Poisson bindings, while projection scaling also works with replay bindings. Metrics retain both the requested mapping and its resolved duration, rate, and projection factors.

The command line exposes named scaling with repeatable `--scale-projection ID=FACTOR`. Existing `--t-ms` and `--input-rate` arguments define generated Poisson execution at request construction. `--inference-timestep-ms` recompiles a request-local graph copy, preserving the original Poisson duration while changing its step count. Metrics record source and effective graph digests and the resolved timestep.

Inference interventions

```
options={"inference_interventions": [
  {
    "kind": "drop_spikes",
    "population_id": "sensory_ping_E",
    "probability": 0.25,
    "seed": 17,
  },
  {
    "kind": "add_poisson_spikes",
    "population_id": "sensory_ping_E",
    "rate_hz": 5.0,
    "seed": 18,
  },
]}
```

The ordered contract uses schema `tools/snn.inference-interventions/v1`. Deletion independently removes each emitted spike with the declared probability. Addition takes the union with a Bernoulli-discretized homogeneous Poisson stream whose probability is `rate_hz * dt_seconds`. Exact population identifiers are required; duplicate kind/target pairs, unknown

fields, non-spiking populations, invalid probabilities, and rates above the timestep probability boundary are rejected.

Each intervention uses a seed-derived stream keyed by its list position, kind, population, and absolute execution step. A continued runtime therefore consumes the same intervention samples as one uninterrupted run. The modified spikes enter normal downstream propagation, delay histories, recordings, and readouts. Metrics retain the requested list and resolved per-step probabilities. The CLI preserves list order through repeatable `--intervention drop:POPULATION=PROBABILITY` and `--intervention add:POPULATION=RATE_HZ` arguments.

Inference artifacts and cache validation

```
manifest = write_inference_artifacts(
  "inference-run", result, graph=graph, seed=17,
)
validate_inference_artifacts(
  "inference-run", graph=graph, seed=17,
)
```

Graph CLI runs persist `recordings.npz`, `outputs.npz`, `parameters.npz`, and `metrics.json` beside `inference-manifest.json`. Schema `tools/snn.inference-artifacts/v1` inventories every NPZ array name, shape, and data type and authenticates each payload by SHA-256. Its graph digest prevents reuse against a different source graph. Its request digest covers the seed, execution protocol, checkpoint, overrides, interventions, recording profile, resolved device, and source/effective graph identities.

`validate_inference_artifacts` verifies the manifest identity, exact file set, payload digests, NPZ inventories, request digest, and optional expected graph and seed. A cache consumer must validate before reuse; paths or directory names are not identities. The standard accuracy/rate/raster conversion below consumes these stable named payloads; scientific interpretation remains at the experiment or campaign layer.

Derived inference products

```
summary = derive_inference_products(
  "inference-run",
  "derived-run",
  logits_id="class_logits",
  labels=labels,
  spike_recordings=("sensory_ping_E.spikes", "sensory_ping_I.spikes"),
)
validate_derived_inference_products(
  "derived-run",
  source_artifact_digest=source_manifest["artifact_digest"],
)
```

Schema `tools/snn.derived-inference/v1` writes exact integer labels and predictions, JSON accuracy and timing, per-presentation per-cell rates in Hz, and sparse raster coordinates for each requested public spike recording. Raster arrays explicitly store zero-based `steps`,

`batches`, `cells`, and the original `[time, batch, cells]` shape. Rates divide each presentation's cell counts by the resolved physical duration.

Derivation first validates the source inference cache, then requires floating `[batch, classes]` logits, integral in-range `[batch]` labels, positive protocol duration, and binary `[time, batch, cells]` recordings with the same batch axis. Its own manifest authenticates every derived payload and links it to the source artifact digest. Scientific thresholds and campaign acceptance decisions remain outside this generic conversion.

Code map

- `tools/snnlang/core.py`: authoring objects and primitive specifications.
- `components.py`: reusable graph-building components.
- `readouts.py` and `ops.py`: standard operations.
- `training.py`: training recipes.
- `compiler.py`: validation and bundle writing.
- `visualize.py`: bundle reports.
- `tools/snn/execution.py`: graph planning and execution.
- `tools/snn/conformance.py`: versioned layered comparison reports.
- `tools/snn/bundle.py`: the narrow legacy adapter.
- `tools/snnlang/tests` and `tools/snn/tests/test_execution.py`: focused conformance fixtures.